

IMDB Top 1000 Movies — SQL Analysis

Table creation and exploratory queries on the IMDB Top 1000 Movies dataset (PostgreSQL syntax)

```
-- =====
-- IMDB Top 1000 Movies — SQL Analysis
-- Table creation + exploratory queries
-- =====

DROP TABLE IF EXISTS MOVIES_RATINGS;

CREATE TABLE IF NOT EXISTS MOVIES_RATINGS
(
    Poster_Link    VARCHAR(300),
    Series_Title   VARCHAR(100),
    Released_Year  VARCHAR(10),
    Certificate     VARCHAR(10),
    Runtime        VARCHAR(10),
    Genre          VARCHAR(50),
    IMDB_Rating    DECIMAL,
    Overview       VARCHAR(500),
    Meta_score     INT,
    Director       VARCHAR(100),
    Star1          VARCHAR(100),
    Star2          VARCHAR(100),
    Star3          VARCHAR(100),
    Star4          VARCHAR(100),
    No_of_Votes    INT,
    Gross          VARCHAR(20) -- stored as text in source CSV (contains commas, e.g. "28,341
,469")
);

SELECT * FROM MOVIES_RATINGS;

-- -----
-- Basic listings
-- -----

SELECT Series_Title, IMDB_Rating
FROM MOVIES_RATINGS
ORDER BY IMDB_Rating DESC;

SELECT Genre, COUNT(*) AS total_movies
FROM MOVIES_RATINGS
GROUP BY Genre
ORDER BY total_movies DESC;

SELECT Director, ROUND(AVG(IMDB_Rating), 2) AS avg_rating
FROM MOVIES_RATINGS
GROUP BY Director
HAVING COUNT(*) >= 3
ORDER BY avg_rating DESC;

SELECT Series_Title, Released_Year, COUNT(*) AS total_movies
FROM MOVIES_RATINGS
GROUP BY Released_Year, Series_Title
ORDER BY Released_Year;

SELECT Series_Title, Released_Year
FROM MOVIES_RATINGS
WHERE Released_Year > '2010'
ORDER BY Released_Year DESC;

SELECT Series_Title, IMDB_Rating
FROM MOVIES_RATINGS
WHERE IMDB_Rating > 8.5
ORDER BY IMDB_Rating DESC;

SELECT Director, COUNT(*) AS movie_count
FROM MOVIES_RATINGS
GROUP BY Director
```

```

ORDER BY movie_count DESC
LIMIT 10;

SELECT Series_Title, No_of_Votes
FROM MOVIES_RATINGS
ORDER BY No_of_Votes DESC
LIMIT 10;

-- -----
-- Gross revenue (numeric cast required – column stored as text)
-- -----

SELECT Series_Title, CAST(REPLACE(Gross, ',', '')) AS gross_usd
FROM MOVIES_RATINGS
WHERE Gross IS NOT NULL
  AND CAST(REPLACE(Gross, ',', '')) AS BIGINT) > 100000000
ORDER BY gross_usd DESC;

SELECT Series_Title, CAST(REPLACE(Gross, ',', '')) AS gross_usd
FROM MOVIES_RATINGS
WHERE Gross IS NOT NULL
ORDER BY gross_usd DESC
LIMIT 5;

SELECT Series_Title, Gross
FROM MOVIES_RATINGS
WHERE Gross IS NULL;

-- -----
-- Runtime
-- -----

SELECT Series_Title, Genre,
  ROUND(AVG(CAST(REPLACE(Runtime, ' min', '')) AS INT)), 2) AS avg_runtime
FROM MOVIES_RATINGS
GROUP BY Series_Title, Genre
ORDER BY avg_runtime DESC;

SELECT DISTINCT Series_Title, Runtime
FROM MOVIES_RATINGS
ORDER BY CAST(REPLACE(Runtime, ' min', '')) AS INT) DESC
LIMIT 5;

-- -----
-- Director / actor lookups
-- -----

SELECT Series_Title, Director
FROM MOVIES_RATINGS
WHERE Director = 'Steven Spielberg';

SELECT Star1 AS actor_name, COUNT(*) AS appearance_count
FROM MOVIES_RATINGS
GROUP BY Star1
ORDER BY appearance_count DESC
LIMIT 10;

SELECT DISTINCT Series_Title
FROM MOVIES_RATINGS
WHERE 'MADHAVAN' IN (UPPER(Star1), UPPER(Star2), UPPER(Star3), UPPER(Star4));

-- -----
-- Above-average ratings & window function
-- -----

SELECT Series_Title, IMDB_Rating
FROM MOVIES_RATINGS
WHERE IMDB_Rating > (SELECT AVG(IMDB_Rating) FROM MOVIES_RATINGS)
ORDER BY IMDB_Rating DESC;

SELECT Series_Title, Released_Year, IMDB_Rating,
  RANK() OVER (PARTITION BY Released_Year ORDER BY IMDB_Rating DESC) AS year_rank
FROM MOVIES_RATINGS;

```

```

-- Average rating by decade (Released_Year cast to INT first)
SELECT FLOOR(CAST(Released_Year AS INT) / 10) * 10 AS decade,
       ROUND(AVG(IMDB_Rating), 2) AS avg_rating
FROM MOVIES_RATINGS
GROUP BY FLOOR(CAST(Released_Year AS INT) / 10) * 10
ORDER BY avg_rating DESC;

-- =====
-- Certificate / Genre filters
-- =====

SELECT Series_Title, Released_Year, Director FROM MOVIES_RATINGS;

SELECT Series_Title, Released_Year, IMDB_Rating FROM MOVIES_RATINGS;

SELECT Series_Title, Released_Year, IMDB_Rating, Certificate
FROM MOVIES_RATINGS
WHERE Certificate IN ('UA', 'A');

SELECT Series_Title, Released_Year, IMDB_Rating, Certificate, Genre
FROM MOVIES_RATINGS
WHERE Certificate IN ('UA', 'A')
   AND IMDB_Rating > 8.3
   AND Genre ILIKE '%DRAMA%';

SELECT DISTINCT *
FROM MOVIES_RATINGS
WHERE Certificate IN ('UA', 'A')
   AND Genre ILIKE '%DRAMA%'
   AND IMDB_Rating > 7.0
   AND 'MADHAVAN' IN (UPPER(Star1), UPPER(Star2), UPPER(Star3), UPPER(Star4))
   AND Released_Year BETWEEN '2000' AND '2020'
ORDER BY Released_Year;

-- Count movies by certificate
SELECT Certificate, COUNT(*) AS total_movies
FROM MOVIES_RATINGS
GROUP BY Certificate;

-- Average rating per genre
SELECT Genre, CAST(AVG(IMDB_Rating) AS NUMERIC(5,2)) AS avg_rating
FROM MOVIES_RATINGS
GROUP BY Genre
ORDER BY avg_rating DESC;

-- Movies released between years
SELECT DISTINCT Series_Title, Released_Year
FROM MOVIES_RATINGS
WHERE Released_Year BETWEEN '1920' AND '2010'
ORDER BY Released_Year;

-- Duplicate title check
SELECT Series_Title, COUNT(*)
FROM MOVIES_RATINGS
GROUP BY Series_Title
HAVING COUNT(*) > 1;

-- Highest rated movie per year
SELECT Released_Year, MAX(IMDB_Rating) AS max_rating
FROM MOVIES_RATINGS
GROUP BY Released_Year;

-- Drama genre with high rating
SELECT Series_Title, IMDB_Rating, Genre, Released_Year
FROM MOVIES_RATINGS
WHERE Genre ILIKE '%DRAM%'
   AND IMDB_Rating > 7;

-- Certificate counts (non-null)
SELECT COUNT(Certificate) AS certificate_count, Certificate
FROM MOVIES_RATINGS
WHERE Certificate IS NOT NULL
GROUP BY Certificate;

```

```

-- Certificate counts above threshold
SELECT COUNT(Certificate) AS certificate_count, Certificate
FROM MOVIES_RATINGS
GROUP BY Certificate
HAVING COUNT(Certificate) > 5
ORDER BY certificate_count DESC;

SELECT COUNT(No_of_Votes) AS total_votes, Certificate
FROM MOVIES_RATINGS
GROUP BY Certificate
ORDER BY Certificate;

-- -----
-- Min / Max rating (UNION examples)
-- -----

SELECT MAX(IMDB_Rating) FROM MOVIES_RATINGS;
SELECT MIN(IMDB_Rating) FROM MOVIES_RATINGS;

SELECT MIN(IMDB_Rating) AS min_rating, MAX(IMDB_Rating) AS max_rating
FROM MOVIES_RATINGS;

SELECT Series_Title, IMDB_Rating
FROM MOVIES_RATINGS
WHERE IMDB_Rating = 9.3;

-- Min and max as two rows
SELECT MIN(IMDB_Rating) AS rating FROM MOVIES_RATINGS
UNION
SELECT MAX(IMDB_Rating) FROM MOVIES_RATINGS;

-- Min and max labeled
SELECT MIN(IMDB_Rating) AS rating, 'MIN_VALUE' AS label
FROM MOVIES_RATINGS
UNION
SELECT MAX(IMDB_Rating) AS rating, 'MAX_VALUE' AS label
FROM MOVIES_RATINGS;

-- =====
-- Additional filtering & string functions
-- =====

-- 1. Basic filtering
SELECT Series_Title, IMDB_Rating, Released_Year
FROM MOVIES_RATINGS
WHERE IMDB_Rating >= 9.0
ORDER BY IMDB_Rating DESC;

-- 2. Multiple conditions
SELECT Series_Title, Genre, Director
FROM MOVIES_RATINGS
WHERE Genre LIKE '%Drama%'
  AND Released_Year >= '2010'
  AND IMDB_Rating > 8.5
ORDER BY No_of_Votes DESC
LIMIT 10;

-- 3. String functions
SELECT
  Series_Title,
  LENGTH(Overview) AS overview_length,
  SUBSTR(Overview, 1, 100) || '...' AS short_overview
FROM MOVIES_RATINGS
WHERE Overview LIKE '%redemption%' OR Overview LIKE '%love%';

```